

FanVerdict Private Leagues

A proposed feature allowing participants to create private leagues inside an existing tournament, invite friends, process join requests, and preserve league history.

Item	Recommendation
Feasibility	Feasible, medium-large scope. Best implemented as a league layer over existing tournaments.
Core constraint	Votes, polls, results, and points should remain tournament-owned. Private leagues should filter standings by league membership.
Highest risk	RLS/security and invite acceptance flow, especially for new users who sign up from an email token.
Best next move	Build in phases: data model + in-app leagues first, then email invites, then historical snapshots.

Executive Assessment

Bottom line

The idea is strong and fits FanVerdict naturally. The safe architecture is not to duplicate tournaments or polls. A private league should be a membership group attached to a source tournament, with its own owner/admins and a standings view filtered to league members.

- This preserves one source of truth for poll locks, votes, results, and scoring.
- It gives participants social groups without fragmenting the tournament data.
- It allows multiple private leagues to coexist inside the same tournament.
- It supports both active and historical views if membership history or final snapshots are captured.

Current Architecture Fit

The current application already has tournaments, tournament members, polls, poll options, votes, points ledger, profiles, email reminders, live standings, and historical standings. That gives this feature a good foundation.

Existing area	How private leagues should use it
tournaments	Private leagues attach to one tournament through tournament_id.
tournament_members	A private league member must also be a member of the source tournament.
polls / votes	No duplicate polls or votes. League standings read the same votes.
points_ledger	League standings sum existing tournament points for active league members.
profiles	Existing users can be invited or approved without exposing email lists broadly.

Existing area	How private leagues should use it
historical standings	Past private leagues need either durable membership history or final snapshots.

Recommended Product Rules

- Any signed-in participant can create a private league for an active or upcoming tournament if they are already a tournament member.
- The creator becomes owner/admin of that private league only, not a tournament admin.
- Private league admins can invite users, approve requests, remove league members, and rename/archive their own league.
- Private league admins cannot create polls, settle results, adjust points, or change tournament membership outside invite/accept behavior.
- An accepted private league invite should enroll the user into the source tournament if they are not already a tournament member.
- A discoverable private league can appear in the tournament directory. An unlisted league should only be reachable through invite link or direct membership.
- League standings should include only active private league members and the same scoring rules as the source tournament.

Proposed Data Model

Proposed private league data layer

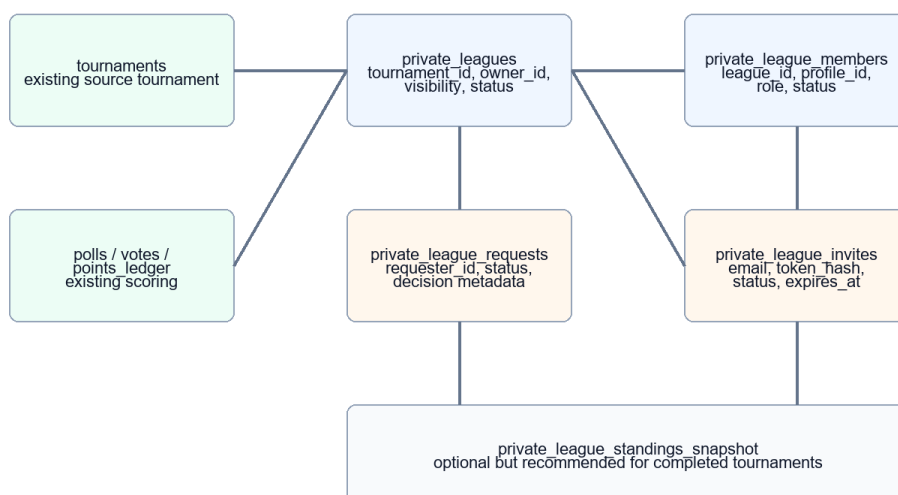
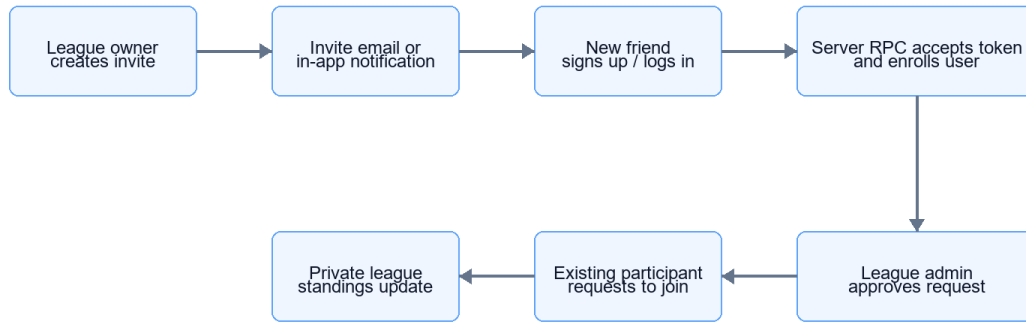


Figure 1. Private leagues as a scoped layer over the existing tournament model.

Table / view	Purpose
private_leagues	League metadata: tournament_id, name, owner_id, visibility, status, starts_at/ends_at inherited or copied from tournament.
private_league_members	League membership: profile_id, role, status, joined_at, removed_at. Unique league_id + profile_id.
private_league_invites	Email or user-targeted invite state: invited_email, invited_profile_id, token_hash, status, expires_at, invited_by.
private_league_join_requests	Request queue: requester_profile_id, note, status, decided_by, decided_at.
private_league_standings view	Live filtered standings derived from existing votes and points ledger.
private_league_standings_snapshot	Recommended archive of final rankings at tournament close.

Key User Flows

Invite and join flow



Key rule: the acceptance/approval operation must be transactional: base tournament membership + private league membership.

Figure 2. Invite and request flows converge into one transactional enrollment operation.

Create League

- 1 Participant opens Private Leagues from the tournament dashboard.
- 2 Chooses Create League, enters league name, visibility, and optional description.
- 3 Adds existing tournament participants by in-app invite.
- 4 Adds new friends by email invite.
- 5 System creates league and owner membership in one transaction.

Accept Invite

- 1 Recipient opens invite link.
- 2 If not signed in, they sign up or log in through normal FanVerdict auth.
- 3 After auth, the token is accepted by a server route or Supabase RPC.
- 4 The user is inserted into `tournament_members` if needed, then `private_league_members`.
- 5 The league appears on their dashboard immediately.

Request Join

- 1 Participant opens All Private Leagues for the selected tournament.
- 2 They request to join a discoverable league.
- 3 League admin sees pending request.
- 4 Approval inserts or activates `private_league_members`, and `tournament_members` if needed.

Revised Screen Designs

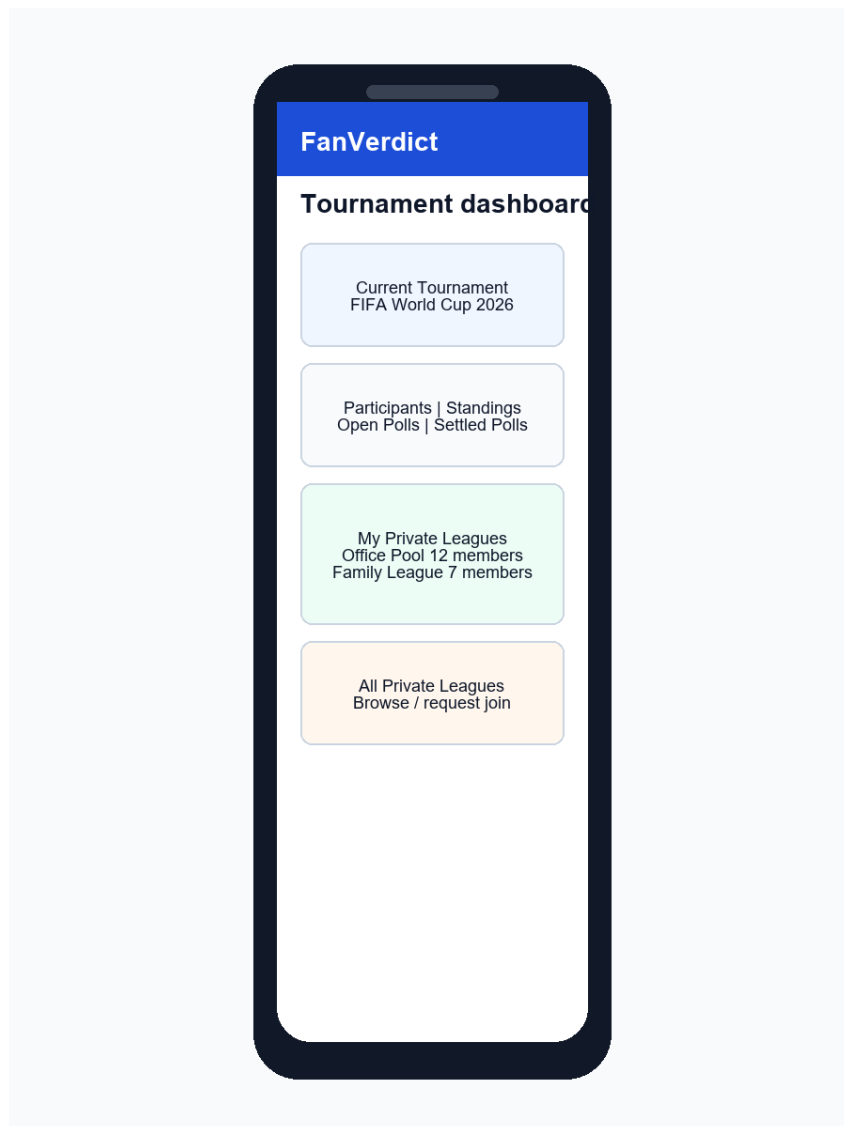


Figure 3. Tournament dashboard adds My Private Leagues and All Private Leagues entry points.

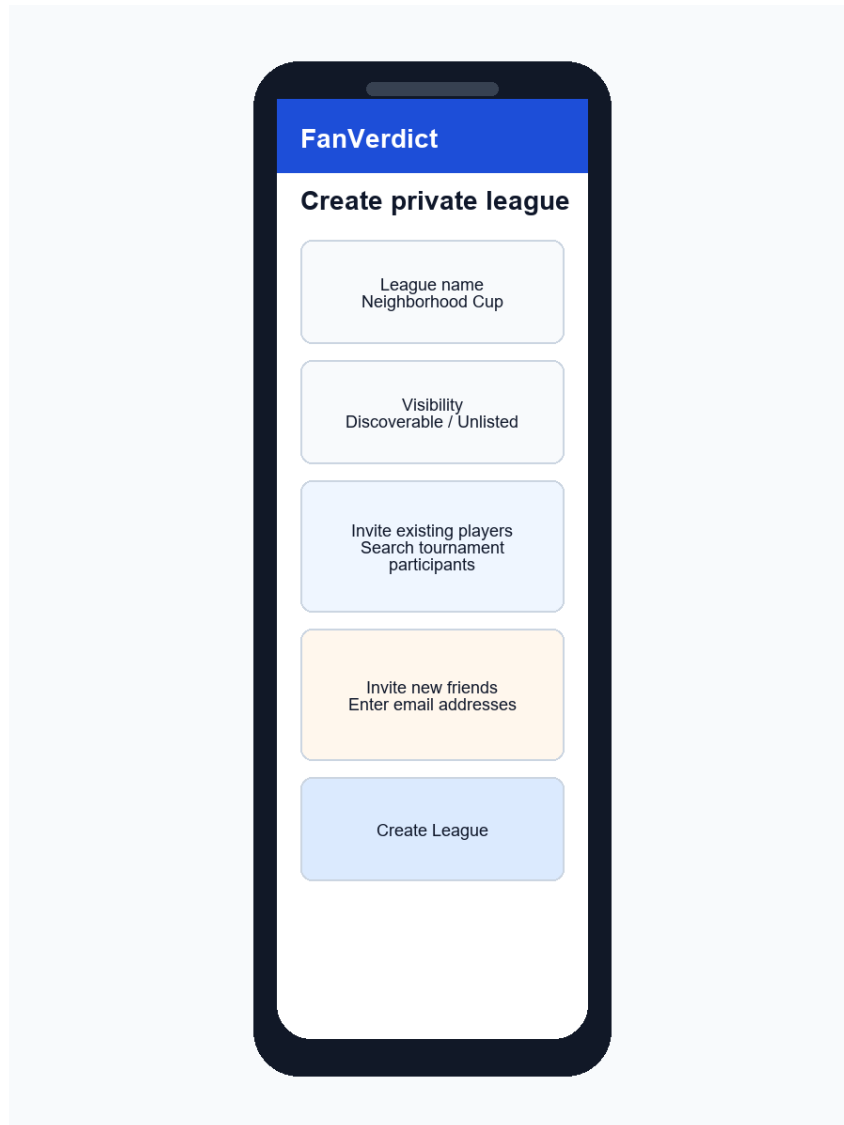


Figure 4. Create flow supports visibility, existing participants, and new-email invites.

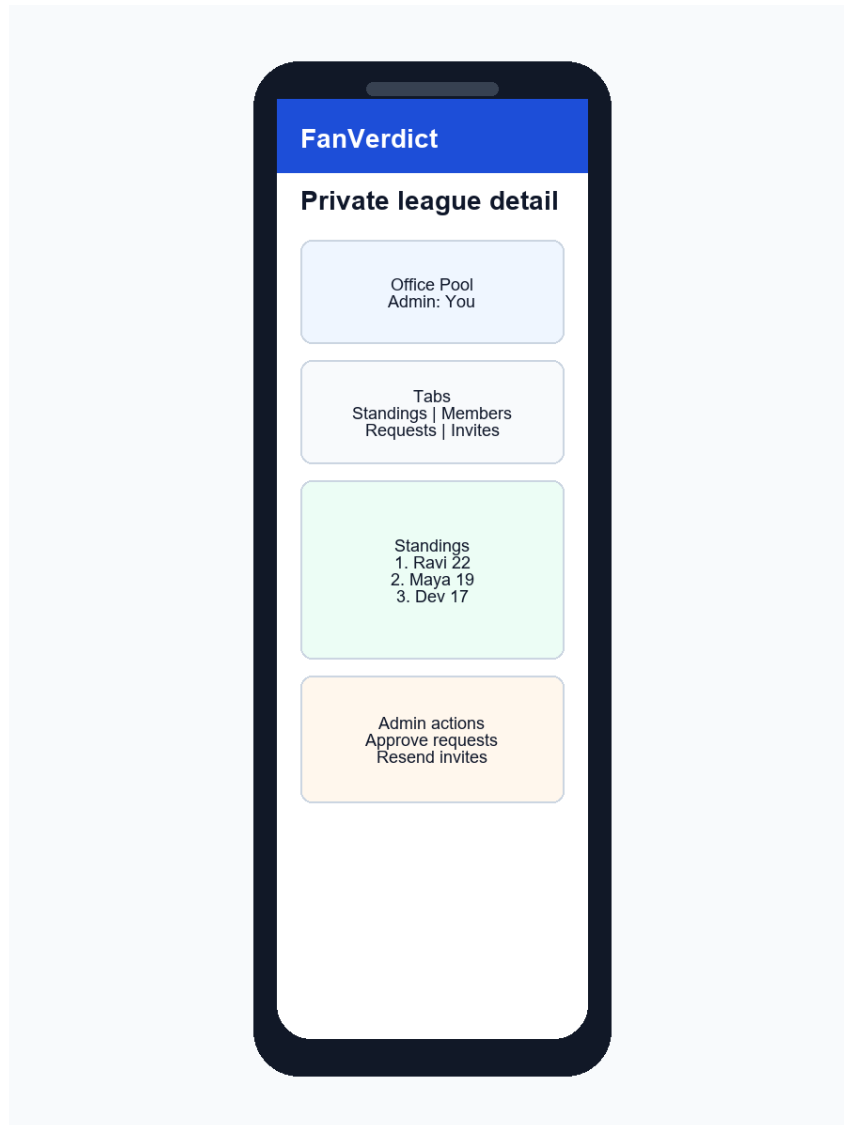


Figure 5. Private league detail separates standings, members, requests, and invites.

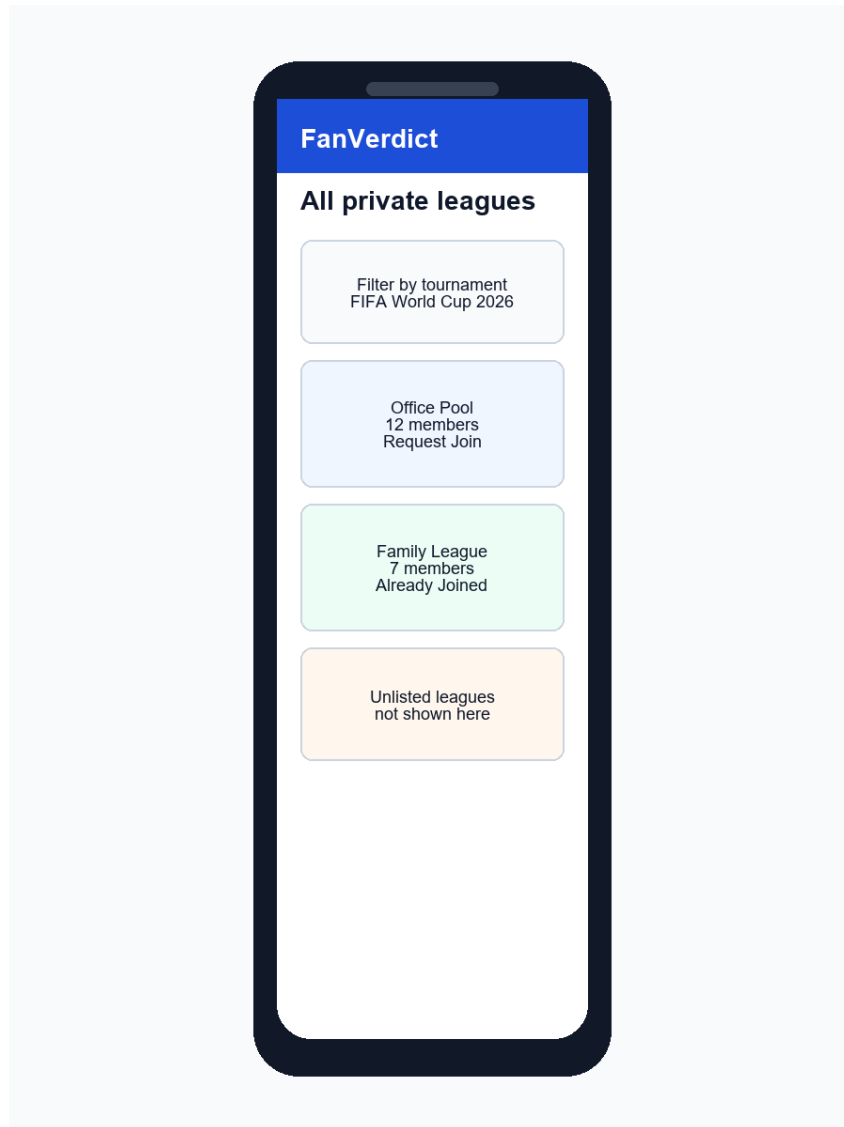


Figure 6. Directory lists discoverable leagues and supports request-to-join.

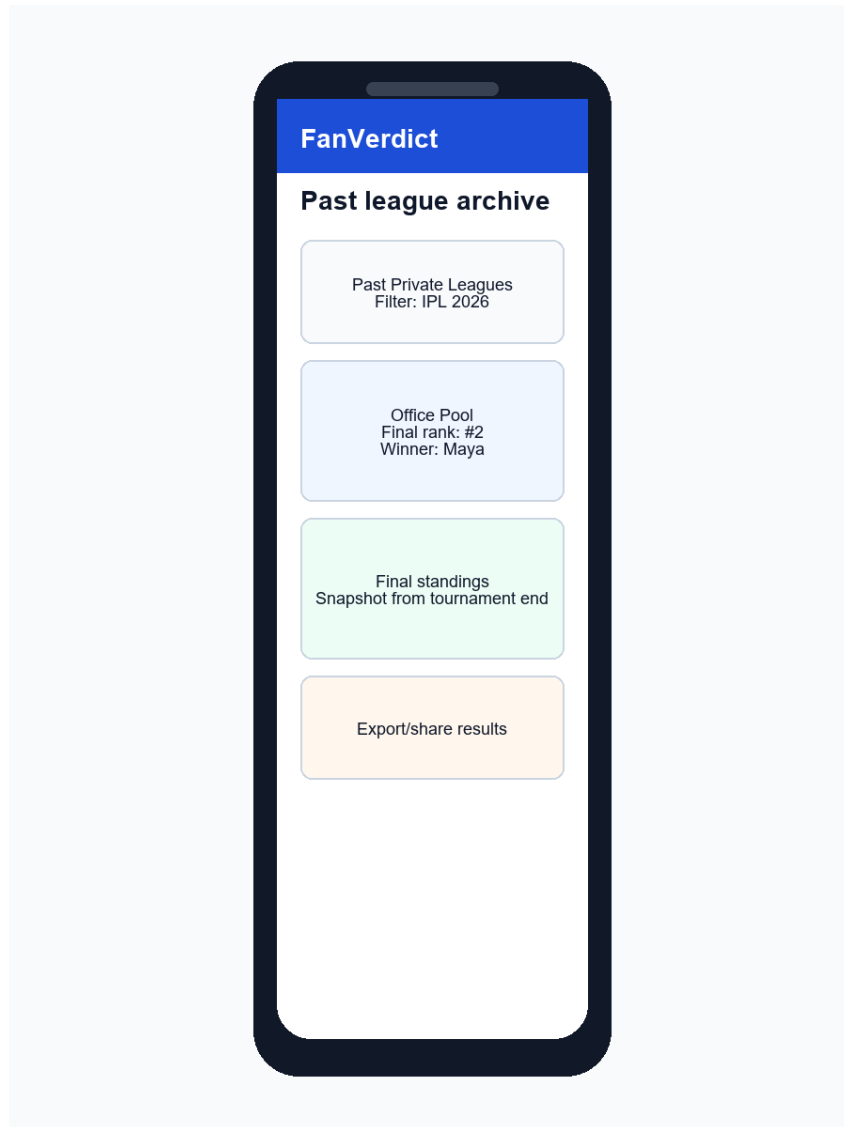


Figure 7. Past participation view relies on final standings snapshots.

Flaws And Fixes

Issue	Why it matters	Recommended fix
Private vs listed leagues	Showing all leagues can conflict with privacy expectations.	Add visibility: discoverable and unlisted. Only discoverable leagues appear in the directory.
Forced addition of existing users	Owners could add people to leagues they did not consent to join.	Use invitations for existing users. Direct add only if privacy settings explicitly allow it.
Duplicate scoring	Copying polls or votes would create inconsistencies.	Reuse tournament polls, votes, and ledger. Calculate league standings by filtering members.
Invite spam	Email invites can be abused and may hit spam.	Rate-limit invites, expire tokens, add resend limits, and use in-app invites for existing users.

Issue	Why it matters	Recommended fix
New user acceptance	Signup is asynchronous and email verification may interrupt token flow.	Persist invite token in the callback route and complete acceptance only after authentication.
Historical accuracy	Removed members or late joiners can change past standings if computed live forever.	Store membership effective dates and create final standings snapshots at tournament completion.
RLS complexity	Client-side Supabase writes may expose privilege mistakes.	Use security-definer RPCs or Next API routes for create, invite, approve, accept, remove.
Private admin scope	League owners should not become tournament admins.	Separate league roles from tournament roles and enforce with dedicated RLS helpers.
Email privacy	Searching all FanVerdict users by email/display name can leak information.	Search only current tournament participants; invite everyone else by exact email entry.
Dashboard clutter	A participant could belong to many leagues.	Show a compact My Leagues panel with filters, pinned leagues, and archive separation.

Security And RLS Requirements

- A user can read a private league if they are a member, the league is discoverable, or they are a tournament/app admin.
- Only private league owners/admins can approve requests, revoke invites, remove members, or change visibility.
- Accepting an invite must validate token hash, expiry, status, target email/profile if present, and tournament status.
- Request approval must be transactional and idempotent.
- League owner/admin role must not grant access to tournament poll management.
- Audit important actions: create league, invite sent, invite accepted, request approved/rejected, member removed, visibility changed.

Implementation Plan

Phase	Scope	Notes
1. Product decisions	Finalize visibility rules, invite behavior, limits, and archive behavior.	Do this before coding because it affects schema and RLS.
2. Schema + RLS	Add private league tables, helper functions, RPCs, and basic types.	Most important technical risk.
3. In-app league management	Create league, my leagues, league detail, standings, members.	No email dependency yet.
4. Join requests	Directory of discoverable leagues, request queue, approve/reject.	Good MVP social loop.
5. Email invites	Invite by email, token route, signup continuation, acceptance.	Higher risk because email deliverability is already a known issue.
6. Historical archive	Membership effective dates and final standings snapshots.	Needed before relying on past private league views.
7. Notifications and polish	In-app alerts, email copy, empty states, admin audit views.	Improves usability and moderation.

MVP Recommendation

Build this as two releases

Release 1 should support private league creation, member invites for existing tournament participants, filtered standings, and join requests. Release 2 should add email invites for new friends, signup-token acceptance, and historical snapshots.

This staged approach keeps the first implementation useful without tying the feature to the hardest email/signup edge cases.

Open Product Questions

- Can any tournament participant create unlimited private leagues, or should there be a per-user/per-tournament limit?
- Should private league admins be allowed to remove members after voting has started?
- Should late joiners be ranked from all tournament points or only points after they joined the private league?
- Should a league be discoverable by default or unlisted by default?
- Should a rejected join request be allowed to request again?
- Should private league names/descriptions be moderated by app admins?
- Should private league standings support export/share links?

Decision

Proceed with design refinement. The feature is feasible and aligns with FanVerdict, but the implementation should be intentionally scoped. The biggest design correction is to treat private leagues as groups inside a tournament, not separate tournaments. That keeps predictions, scoring, and historical data consistent.